Code Appendix — Corporate Bankruptcy Risk Prediction with Machine Learning

Berke Pehlivan
Machine Learning for Finance
University of Bonn

Academic Year 2025/2026

This appendix contains selected implementation excerpts from the notebook-based project. The full implementation is available in the six numbered notebooks. Some excerpts are shortened to keep the appendix readable. The research paper contains the methodology and results; this appendix documents the main implementation choices behind them.

1 Company-level train/test split

Complete companies are assigned either to the training side or to the test side. This prevents the same company from appearing in both sets and avoids direct company leakage.

Source: *notebooks/02_company_split_and_eda.ipynb* *and* *notebooks/03_model_training_and_validation.ipynb*

```python
def create_company_level_split(
    data: pd.DataFrame,
    test_size: float = TEST_SIZE,
    random_state: int = RANDOM_STATE,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    required_columns = {COMPANY_COLUMN, TARGET_COLUMN}
    missing_columns = required_columns.difference(data.columns)
    if missing_columns:
        msg = f"Missing required split columns: {sorted(missing_columns)}"
        raise ValueError(msg)

    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=test_size,
        random_state=random_state,
    )

    train_idx, test_idx = next(
        splitter.split(
            data,
            y=data[TARGET_COLUMN],
            groups=data[COMPANY_COLUMN],
        )
    )

    train_data = data.iloc[train_idx].copy()
    test_data = data.iloc[test_idx].copy()
    return train_data, test_data


def check_no_company_overlap(train_data: pd.DataFrame, test_data: pd.DataFrame) -> bool:
    train_companies = set(train_data[COMPANY_COLUMN].unique())
    test_companies = set(test_data[COMPANY_COLUMN].unique())
    return train_companies.isdisjoint(test_companies)
```

2 Model dataset and target separation

The binary target is created from the raw status label. Company and year are kept for splitting and diagnostics, but they are not financial predictors. The feature columns are identified before the binary target is added, which prevents the target from being duplicated as a predictor.

Source: *notebooks/01_data_audit_and_preparation.ipynb*

```python
def encode_target(data: pd.DataFrame) -> pd.DataFrame:
    target_mapping = {
        ALIVE_LABEL: 0,
        FAILED_LABEL: 1,
    }
```

```
    data = data.copy()
    data[TARGET_COLUMN] = data[RAW_TARGET_COLUMN].map(target_mapping).astype(int)
    return data


def create_model_dataset(raw_data: pd.DataFrame) -> tuple[pd.DataFrame, list[str]]:
    validate_required_columns(raw_data)
    validate_target_labels(raw_data)

    feature_columns = identify_feature_columns(raw_data)
    data = encode_target(raw_data)

    constant_columns = find_constant_columns(data, feature_columns)
    retained_feature_columns = [
        column for column in feature_columns if column not in constant_columns
    ]

    selected_columns = [
        COMPANY_COLUMN,
        YEAR_COLUMN,
        TARGET_COLUMN,
        *retained_feature_columns,
    ]

    model_dataset = data[selected_columns].copy()
    return model_dataset, constant_columns
```

3 Logistic Regression pipeline

Logistic Regression is fitted inside a scikit-learn pipeline. The scaler is learned from the fitting data. Balanced class weighting gives failed observations more influence during fitting. It does not rebalance the test set or directly set the classification threshold.

Source: notebooks/03_model_training_and_validation.ipynb

```
def build_logistic_pipeline(
    penalty: str = "l2",
    c_value: float = 1.0,
    class_weight: str | dict | None = "balanced",
) -> Pipeline:
    if penalty not in {"l1", "l2"}:
        msg = "Only 'l1' and 'l2' penalties are supported in this project."
        raise ValueError(msg)

    l1_ratio = 1.0 if penalty == "l1" else 0.0

    model = LogisticRegression(
        C=c_value,
        l1_ratio=l1_ratio,
        solver="saga",
        class_weight=class_weight,
        max_iter=5_000,
        random_state=42,
    )

    return Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("model", model),
        ]
    )
```

4 Gradient Boosting validation selection

Gradient Boosting is selected using internal validation PR-AUC, implemented as scikit-learn average precision. The final test set is not used in this selection step.

Source: notebooks/03_model_training_and_validation.ipynb

```
def select_gradient_boosting(
    x_train: pd.DataFrame,
    y_train: pd.Series,
    x_valid: pd.DataFrame,
    y_valid: pd.Series,
) -> tuple[GradientBoostingClassifier, dict[str, Any], float]:
    parameter_grid = {
        "n_estimators": [100, 150],
        "learning_rate": [0.03, 0.05],
        "max_depth": [2, 3],
        "min_samples_leaf": [100],
    }

```

```
    sample_weight = compute_sample_weight(class_weight="balanced", y=y_train)
    best_model = None
    best_params = None
    best_score = -1.0

    for n_estimators, learning_rate, max_depth, min_samples_leaf in product(
        parameter_grid["n_estimators"],
        parameter_grid["learning_rate"],
        parameter_grid["max_depth"],
        parameter_grid["min_samples_leaf"],
    ):
        candidate = build_gradient_boosting(
            n_estimators=n_estimators,
            learning_rate=learning_rate,
            max_depth=max_depth,
            min_samples_leaf=min_samples_leaf,
        )
        candidate.fit(x_train, y_train, sample_weight=sample_weight)

        probability_failed = candidate.predict_proba(x_valid)[:, 1]
        score = average_precision_score(y_valid, probability_failed)

        if score > best_score:
            best_model = candidate
            best_params = {
                "n_estimators": n_estimators,
                "learning_rate": learning_rate,
                "max_depth": max_depth,
                "min_samples_leaf": min_samples_leaf,
            }
            best_score = score

    return best_model, best_params, best_score
```

5 Final-test evaluation

The final-test stage runs after the modelling choices are fixed. The main classification comparison uses each fitted model's default `predict` rule. Failed-class scores are also saved for threshold-independent metrics such as PR-AUC and ROC-AUC.

Source: notebooks/04_final_results_and_interpretation.ipynb

```
def evaluate_models_on_dataset(
    models: dict[str, object],
    data: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    x_test, y_test = split_features_target(data)

    metric_rows = []
    report_tables = []
    prediction_tables = []

    for model_name, model in models.items():
        y_pred = model.predict(x_test)
        probability_failed = get_probability_failed(model, x_test)

        metric_rows.append(
            evaluate_binary_classifier(
                model_name=model_name,
                y_true=y_test,
                y_pred=y_pred,
                probability_failed=probability_failed,
            )
        )

        report_tables.append(
            create_classification_report_table(
                model_name=model_name,
                y_true=y_test,
                y_pred=y_pred,
            )
        )

        prediction_tables.append(
            create_prediction_table(
                validation_data=data,
                model_name=model_name,
                y_pred=y_pred,
                probability_failed=probability_failed,
            )
        )

    return (
        pd.DataFrame(metric_rows),
        pd.concat(report_tables, ignore_index=True),
        pd.concat(prediction_tables, ignore_index=True),
    )
```

6 Validation-only threshold analysis

Threshold scenarios are created from validation predictions only. They show how precision, recall, and F1 change when the score cutoff changes. They are not production recommendations, and they are not selected using the final test set.

Source: notebooks/05_threshold_and_pca_extensions.ipynb

```python
def create_threshold_analysis(
    predictions: pd.DataFrame,
    thresholds: list[float] | None = None,
    model_names: list[str] | None = None,
) -> pd.DataFrame:
    if thresholds is None:
        thresholds = [value / 100 for value in range(1, 100)]

    if model_names is None:
        model_names = sorted(predictions["model"].unique())

    rows = []
    for model_name in model_names:
        model_predictions = predictions[predictions["model"] == model_name]
        y_true = model_predictions["actual_failed"]
        probability_failed = model_predictions["probability_failed"]

        for threshold in thresholds:
            y_pred = (probability_failed >= threshold).astype(int)
            rows.append(
                {
                    "model": model_name,
                    "threshold": threshold,
                    "precision_failed": precision_score(y_true, y_pred, zero_division=0),
                    "recall_failed": recall_score(y_true, y_pred, zero_division=0),
                    "f1_failed": f1_score(y_true, y_pred, zero_division=0),
                    "predicted_failed_share": float(y_pred.mean()),
                }
            )

    return pd.DataFrame(rows)


def select_thresholds(
    threshold_analysis: pd.DataFrame,
    recall_targets: tuple[float, ...] = (0.6, 0.8),
) -> pd.DataFrame:
    for model_name, model_data in threshold_analysis.groupby("model"):
        best_f1_row = model_data.sort_values(
            ["f1_failed", "precision_failed", "threshold"],
            ascending=[False, False, True],
        ).iloc[0]

        # ... store the maximum-F1 rule ...

        for recall_target in recall_targets:
            feasible = model_data[model_data["recall_failed"] >= recall_target]
            if feasible.empty:
                continue
            best_recall_target_row = feasible.sort_values(
                ["precision_failed", "f1_failed", "threshold"],
                ascending=[False, False, False],
            ).iloc[0]
            # ... store the recall-target scenario ...
```

7 Full reproducible implementation

The complete implementation is available in the repository. From the repository root, the full notebook workflow can be run with:

```
for notebook in notebooks/0*.ipynb; do
    jupyter nbconvert --to notebook --execute --inplace \
        --ExecutePreprocessor.timeout=900 "$notebook"
done
```

The notebook assertions replace the previous automated test suite for this simplified academic version. They are placed next to the analysis steps they protect.